Câu hỏi **1**

Đúng

Đạt điểm 1,00 trên 1,00

Given the description of a program in HLang as follows:

A program in HLang consists of many declarations, which are **variable** and **function declarations**.

Modify the HLang.g4 as follows:

program: // write for program rule here using vardecl and funcdecl

vardecl: 'vardecl' ;

funcdecl: 'funcdecl' ;

WS: [\t\r\n] -> skip;

ERROR_CHAR: . {raise ErrorToken(self.text)};

For example:

Test	Result
"""vardecl"""	successful

Answer: (penalty regime: 0 %)

```
1
```

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	
✓	"""vardecl"""	successful	successful	✓

Câu hỏi **2**

Đúng

Đạt điểm 1,00 trên 1,00

Given the description of a program in HLang as follows:

A program in HLang consists of many declarations, which are **variable** and **function declarations**.

A **variable declaration** starts with a type, which is **int** or **float**, then a comma-separated list of identifiers and ends with a semicolon.

A **function declaration** also start with a type and then an identifier, which is the function name, and then parameter declaration and ends with a body. The parameter declaration starts with a left round bracket '(' and a null-able semicolon-separated list of parameters and ends with a right round bracket ')'. Each parameter always starts with a type and then a comma-separated list of identifier.

Modify HLang.g4 as follows:

program: // write your rule here

//And some other rules for variable declaration, function declaration and other rules

body: 'body';

ID: // includes a sequence of alphabetic characters.

WS: [\t\r\n] -> skip;

ERROR_CHAR: . {raise ErrorToken(self.text)};

For example:

Test	Result
"""int a, b,c; float foo(int a; float c, d) body float goo (float a, b) body"""	successful

Answer: (penalty regime: 0 %)

```
1
```

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

Câu hỏi **3**

Đúng

Đạt điểm 1,00 trên 1,00

Given the description of a program in HLang as follows:

A program in HLang consists of many declarations, which are **variable** and **function declarations**.

A **variable declaration** starts with a type, which is **int** or **float**, then a comma-separated list of identifiers and ends with a semicolon.

A **function declaration** also start with a type and then an identifier, which is the function name, and then parameter declaration and ends with a body. The parameter declaration starts with a left round bracket '(' and a null-able semicolon-separated list of parameters and ends with a right round bracket ')'. Each parameter always starts with a type and then a comma-separated list of identifier. A body starts with a left curly bracket '{', follows by a null-able list of variable declarations or statements and ends with a right curly bracket '}'.

There are **3 kinds of statements**: assignment, call and return. All statements must end with a semicolon. An assignment statement starts with an identifier, then an equal '=', then an expression. A call starts with an identifier and then follows by a null-able comma-separated list of expressions enclosed by round brackets. A return statement starts with a symbol 'return' and then an expression.

Modify HLang.g4 as follows:

program :// write your rule for program here

//And some other rules for variable declaration, function declaration, statements but using following expr for an expression

expr: 'expr';

ID: //includes a sequence of alphabetic characters

WS: [\t\r\n] -> skip;

ERROR_CHAR: . {raise ErrorToken(self.text)};

For example:

Test	Result

"""int a, b,c;
float foo(int a; float c, d) {
  int e ;
  e = expr ;
  c = expr ;
  foo(expr);
  return expr;
}
float goo (float a, b) {
  return expr;
}"""
	successful

Answer: (penalty regime: 0 %)

```
1
```

HLang.interp

HLang.tokens

HLangLexer.interp

HLangLexer.py

HLangLexer.tokens

HLangParser.py

HLangVisitor.py

	Test	Expected	Got	

✓	"""int a, b,c;
	float foo(int a; float c, d) {
	int e ;
	e = expr ;
	c = expr ;
	foo(expr);
	return expr;
	}
	float goo (float a, b) {
	return expr;
	}"""
		successful	successful	✓

✓	"""int a, b,;"""	Error on line 1 col 9: ;	Error on line 1 col 9: ;	✓
✓	"""float foo(int a, float c, d) {}"""	Error on line 1 col 17: float	Error on line 1 col 17: float	✓
✓	"""float foo(int a; float c, d;) {}"""	Error on line 1 col 28:)	Error on line 1 col 28:)	✓
✓	"""int c; c = expr;"""	Error on line 2 col 0: c	Error on line 2 col 0: c	✓

✓	"""int a, b,c;
	float foo(int a; float c, d) {
	int e = expr;
	e = expr ;
	c = expr ;
	return expr;
	}
	float goo(float a, b) {
	return expr;
	}"""
		Error on line 3 col 9: =	Error on line 3 col 9: =	✓

✓	"""int a, b,c;
	float foo(int a; float c, d) {
	int e ;
	e = expr ;
	c = expr ;
	return expr
	}
	"""
		Error on line 7 col 0: }	Error on line 7 col 0: }	✓

Câu hỏi **4**

Đúng

Đạt điểm 1,00 trên 1,00

Given the description of a program in HLang as follows:

A program in HLang consists of many declarations, which are **variable** and **function declarations**.

A **variable declaration** starts with a type, which is **int** or **float**, then a comma-separated list of identifiers and ends with a semicolon.

A **function declaration** also start with a type and then an identifier, which is the function name, and then parameter declaration and ends with a body. The parameter declaration starts with a left round bracket '(' and a null-able semicolon-separated list of parameters and ends with a right round bracket ')'. Each parameter always starts with a type and then a comma-separated list of identifier. A body starts with a left curly bracket '{', follows by a null-able list of variable declarations or statements and ends with a right curly bracket '}'.

There are **3 kinds of statements**: assignment, call and return. All statements must end with a semicolon. An assignment statement starts with an identifier, then an equal '=', then an expression. A call starts with an identifier and then follows by a null-able comma-separated list of expressions enclosed by round brackets. A return statement starts with a symbol 'return' and then an expression.

An **expression** is a construct which is made up of operators and operands. They calculate on their operands and return new value. There are four kinds of infix operators: '+', '-', '*' and '/' where '+' have lower precedence than '-' while '*' and '/' have the highest precedence among these operators. The '+' operator is right associative, '-' is non-associative while '*' and '/' is left-associative. To change the precedence, a sub-expression is enclosed in round brackets. The operands can be an integer literal, float literal, an identifier, a call or a sub-expression.

For example:

```
int a, b, c;
float foo(int a; float c, d) {
     int e;
     e = a + 4;
     c = a * d / 2.0;
     return c + 1;
}
float goo(float a, b) {
     return foo(1, a, b);
}
```

Some tokens:

1. An identifier includes a sequence of alphabetic characters.

2. An integer number includes a sequence of numerical characters.

3. A real (float) number includes two parts: integer and fractional parts. The integer and fractional part are like a integer number, but separated by a point (.).

Your task:

Write a grammar of the program in HLang.g4 using ANTLR and submit its generation files to this exercise.

For example:

Test	Result

"""int a, b,c;
float foo(int a; float c, d) {
  int e ;
  e = a + 4 ;
  c = a * d / 2.0 ;
  return c + 1;
}
float goo (float a, b) {
  return foo(1, a, b);
}"""
	successful